

Human Computer Interface Programming

Practical Exam (ASSIGNMENT 1)

Gym Membership Management System

REST API

Scenario

A fitness company manages gym members, fitness programs, and memberships.

The system should manage:

- Members
- Fitness Programs
- Memberships

Members can enroll in multiple fitness programs and maintain active memberships.

Tech Stack

Java 17 · Spring Boot 3.x · Spring Data JPA · H2 (in-memory) · Maven

Part	Topic	Points
I	Entities and Relationships	25
II	Repository and Service Layer	30
III	REST API, Validation and Security	30
IV	Stream-based Reporting Endpoints	15
Total		100

PART I – Entities and Relationships (25 pts)

1. Entities (10 pts)

Member

- id
- firstName
- lastName
- email
- active

FitnessProgram

- id
- name
- description
- durationWeeks
- capacity

Membership

- id
- startDate
- endDate
- monthlyFee

2. Relationships (15 pts)

Many-To-Many (10 pts)

Member ↔ FitnessProgram

Rules:

- One member can enroll in many fitness programs

- One fitness program can contain many members

Use:

- @ManyToMany
- @JoinTable

One-To-Many (5 pts)

Member → Membership

Rules:

- One member can have multiple memberships
- One membership belongs to one member

Use:

- @OneToMany
- @ManyToOne

PART II – Repository and Service Layer (30 pts)

1. Repository Layer (5 pts)

Create:

- MemberRepository
- FitnessProgramRepository
- MembershipRepository

All extending:

JpaRepository<Entity, Long>

2. DTO Layer (5 pts)

Create DTOs for Member.

CreateMemberRequest

Fields:

- firstName
- lastName
- email
- active

MemberResponse

Fields:

- id
- fullName
- email
- active
- enrolledPrograms

3. Service Implementation (20 pts)

MemberService (10 pts)

List<Member> getAllMembers()

Member getMemberById(Long id)

Member createMember(CreateMemberRequest request)

Member updateMember(Long id, Member member)

void deleteMember(Long id)

```

Member enrollMemberToProgram(Long
memberId, Long programId)

Member removeMemberFromProgram(Long
memberId, Long programId)

List<Membership>
getMemberMemberships(Long memberId)
Business Rules:
    • Email must be unique
    • Inactive members cannot enroll in programs

```

FitnessProgramService (5 pts)

```

List<FitnessProgram> getAllPrograms()

FitnessProgram getProgramById(Long id)

FitnessProgram
createProgram(FitnessProgram program)

FitnessProgram updateProgram(Long id,
FitnessProgram program)

void deleteProgram(Long id)

```

```

int getEnrollmentCount(Long programId)
Business Rules:
    • Capacity must be positive
    • Enrollment count cannot exceed capacity

```

MembershipService (5 pts)

```

List<Membership> getAllMemberships()

Membership getMembershipById(Long id)

Membership createMembership(Long
memberId,
                                Membership
membership)

Membership updateMembership(Long id,
                                Membership
membership)

void deleteMembership(Long id)

List<Membership> getExpiredMemberships()
Business Rules:
    • End date must be after start date
    • Monthly fee must be positive

```

PART III – REST API, Validation and Security (30 pts)

1. REST Controllers (10 pts)

MemberController

Base URL:

/members

Method	Endpoint
GET	/members
GET	/members/{id}
POST	/members
PUT	/members/{id}
DELETE	/members/{id}
POST	/members/{memberId}/programs/{programId}
DELETE	/members/{memberId}/programs/{programId}
GET	/members/{id}/memberships

FitnessProgramController

Base URL:

/programs

Method	Endpoint
GET	/programs
GET	/programs/{id}
POST	/programs
PUT	/programs/{id}
DELETE	/programs/{id}
GET	/programs/{id}/enrollment-count

MembershipController

Base URL:

/memberships

Method	Endpoint
GET	/memberships
GET	/memberships/{id}
POST	/memberships/member/{memberId}
PUT	/memberships/{id}
DELETE	/memberships/{id}
GET	/memberships/expired

2. Validation Requirements (10 pts)

Member Validation

@NotBlank

for:

- firstName
- lastName

@Email

for:

- email

FitnessProgram Validation

@NotBlank

for:

- name

@Positive

for:

- durationWeeks
- capacity

Membership Validation

@Positive

for:

- monthlyFee

Business Rule:

- endDate must be after startDate

Use:

@Valid

for request body validation.

3. Exception Handling (5 pts)

Create:

ResourceNotFoundException

Implement:

@RestControllerAdvice

Handle:

- ResourceNotFoundException
- MethodArgumentNotValidException
- IllegalStateException

Return:

- proper HTTP status codes
- validation messages
- business error messages

4. Security Implementation (5 pts)

Secure all endpoints using:

OncePerRequestFilter

Required header:

Authorization: Bearer gym-secret-token

Exclude:

/h2-console/**

If token is invalid or missing:

401 Unauthorized

PART IV – Stream-based Reporting Endpoints (15 pts)

Create ReportController

Base URL:

/reports

1. Top Active Member Report (5 pts)

Endpoint:

GET /reports/top-member

Return member enrolled in the highest number of fitness programs.

Required Streams:

groupBy()

counting()

max()

2. Membership Revenue Analytics (5 pts)

Endpoint:

GET /reports/membership-revenue

Return:

```
{
  "totalMemberships": 180,
  "averageMonthlyFee": 35.5,
  "highestMonthlyFee": 80,
  "lowestMonthlyFee": 20,
  "totalMonthlyRevenue": 6390
}
```

Required Streams:

mapToDouble()

sum()

average()

max()

min()

3. Program Enrollment Statistics (5 pts)

Endpoint:

GET /reports/program-statistics

Return:

```
{
  "totalPrograms": 25,
  "totalEnrollments": 320,
  "averageEnrollment": 12.8,
  "mostPopularProgram": "CrossFit"
}
```

Required Streams:

averagingInt()

groupBy()

counting()

max()

Grading Summary

Part	Description	Points
Part I	Entities & Relationships	25
Part II	Repository, DTO & Services	30
Part III	Controllers, Validation, Security & Exception Handling	30
Part IV	Stream-based Reporting Endpoints	15
Total	100	

Expected Project Structure

src/main/java/com/example/gymmanagement

```
├── controller
├── dto
├── entity
├── exception
├── repository
├── security
├── service
└── GymMembershipManagementApplication.java
```